

ERNU.EU

IT & SECURITY SOLUTIONS

GHID COMPLET DEVSECOPS CI PENTRU GITHUB

Testarea codului, compliance, securitate, SBOM și review

Implementare open-source, manuală și neblokantă

EDIȚIA 1.1

Include fișiere complete, scripturi, configurații și proceduri

Profil: manual-nonblocking-v1 | GitHub Actions

Cuprins executiv

Acest ghid descrie implementarea de la zero a unui flux DevSecOps open-source într-un proiect GitHub nou. Modelul începe în regim manual și neblocant, produce dovezi tehnice standardizate și permite activarea controlată a Quality Gates după calibrare.

Capitol	Rezultat practic
1. Model operațional	Ce rulează, când rulează și de ce nu blochează dezvoltarea
2. Arhitectură și instrumente	SonarQube, Semgrep, MegaLinter, Trivy, Gitleaks, ORT
3. Implementare GitHub	Structură, permisiuni, branch, workflow_dispatch și artefacte
4. Operare și interpretare	Pornire, triere, excepții, retenție și audit
5. SonarQube self-hosted	Instalare Docker, secrete, backup și integrare
6. Evoluție	Trecerea de la pilot la controale automate obligatorii
Anexe	Codul complet al fișierelor și variante pe stack

Decizie de design

Profilul manual-nonblocking-v1 folosește exclusiv workflow_dispatch. Nu rulează la push sau Pull Request, nu schimbă Branch Protection și nu execută deployment.

1. Obiectiv, principii și limitări

Scopul este introducerea unei verificări coerente pentru calitatea codului, securitate, dependențe, secrete, infrastructură ca cod, licențe și trasabilitatea review-ului. Un singur scanner nu acoperă toate aceste domenii; platforma combină controale complementare.

Principii obligatorii

- least privilege: workflow-ul primește implicit numai contents: read;
- separarea analizei de build și deployment;
- constatările automate sunt validate înainte de a deveni blocante;
- secretele reale nu sunt stocate în Git sau în artefacte;
- orice excepție are justificare, proprietar și termen de expirare;
- review-ul AI este consultativ și nu înlocuiește aprobarea umană;
- versiunile imaginilor și acțiunilor se fixează înainte de profilul blocking.

Limitare importantă

SonarQube Community Build este adecvat pentru baseline și analiza ramurii principale. Funcțiile complete de branch/PR trebuie evaluate în raport cu ediția instalată. Pilotul nu presupune automat existența PR decoration.

Versiunea recomandată pentru replicare

RECOMMENDED-ENTERPRISE-V1

SonarQube Community + Semgrep CE + MegaLinter + Trivy + Gitleaks + Reviewdog reprezintă baza recomandată. Se adaugă ORT pentru conformitate și PR-Agent + Ollama pentru review AI complet self-hosted. Stackul se standardizează într-un workflow reutilizabil și devine obligatoriu pentru toate repository-urile după calibrare.

Aceasta este arhitectura țintă pentru orice repository nou. Profilul manual-nonblocking-v1 rămâne etapa sigură de onboarding; recommended-enterprise-v1 este versiunea recomandată pentru replicare și guvernanta organizațională.

Componentă	Statut recomandat	Rol
SonarQube Community	Obligatory	quality dashboard, maintainability, coverage
Semgrep CE	Obligatory	SAST și reguli proprii
MegaLinter	Obligatory	code style/compliance multi-limbaj
Trivy	Obligatory	CVE, image/repository, secrete și IaC
Gitleaks	Obligatory	secrete în fișiere și istoricul Git
Reviewdog	Obligatory în faza PR	comentarii inline din scanere
ORT	Obligatory la release/compliance	licențe, policy-as-code și SBOM
PR-Agent + Ollama	Recomandat	review AI self-hosted consultativ

Model de distribuire și guvernanta

- Workflow-ul și politicile sunt menținute într-un repository central.
- Repository-urile consumatoare fixează versiunea reutilizabilă la un tag aprobat.
- Onboardingul începe manual și neblocant timp de 2-4 săptămâni.
- Reviewdog este activat când analiza rulează în context de Pull Request.
- ORT rulează la release și periodic, cu politica de licențe aprobată juridic.
- PR-Agent folosește un runner self-hosted și Ollama; recomandările nu decid merge-ul.
- Excluderile au justificare, proprietar, control compensator și expirare.

2. Arhitectura fluxului

Etapă	Control	Dovadă
Checkout	Istoric complet când este necesar	SHA și branch în GitHub run
SAST	Semgrep CE + reguli OWASP	semgrep.sarif
Supply chain	Trivy CVE/secret/misconfig	trivy.sarif
Secret history	Gitleaks pe istoricul Git	gitleaks.sarif
Compliance	MegaLinter multi-limbaj	megalinter-reports
Inventar	CycloneDX SBOM	sbom.cdx.json
Quality dashboard	SonarQube opțional	Quality Gate și istoric

Flux: operatorul pornește auditul manual, fiecare job analizează copia efemeră a repository-ului, rapoartele sunt încărcate ca artefacte timp de 14 zile, iar rezultatele sunt clasificate. Niciun job nu accesează staging sau production.

3. Implementare pe un proiect GitHub nou

3.1 Pregătirea repository-ului

1. Creați repository-ul și ramura implicită main.
2. Adăugați README, licența proiectului, .gitignore și lockfile-urile managerelor de pachete.
3. Creați directoarele .github/workflows și docs.
4. Adăugați workflow-ul manual din anexă și documentul operațional.
5. Publicați modificarea într-un branch feature/manual-devsecops-audit-v1.
6. Deschideți PR, verificați diff-ul și integrați-l.
7. După integrare, porniți Actions > Manual DevSecOps Audit > Run workflow.

3.2 De ce workflow-ul trebuie întâi integrat

GitHub afișează în mod normal un workflow cu workflow_dispatch în pagina Actions după ce definiția există pe ramura implicită. De aceea, implementarea se examinează prin PR, se integrează, apoi se lansează manual. Faptul că este integrat nu îl face automat: triggerul rămâne exclusiv manual.

3.3 Permisuni

Pentru profilul v1 este suficient contents: read. Nu acordați pull-requests: write, security-events: write sau packages: write decât atunci când activați explicit comentarii, upload în Code Scanning ori publicarea imaginilor. Fiecare extindere trebuie documentată prin threat model.

4. Instrumente și responsabilități

Instrument	Responsabilitate	Nu înlocuiește
Semgrep CE	pattern-based SAST și reguli custom	teste și review arhitectural
MegaLinter	stil, formatare și reguli pe limbaje	SAST aprofundat
Trivy	CVE, secrete, IaC și SBOM	validarea logicii aplicației
Gitleaks	secrete în fișiere și istoric Git	rotația secretului compromis
SonarQube	trend, maintainability, coverage și gate	review uman
ORT	licențe, attribution și policy-as-code	decizie juridică
Reviewdog	comentarii pe diff în faza 2	scannerul sursă
PR-Agent/Ollama	review semantic consultativ	control determinist

5. Operare, triere și excepții

Pornirea auditului

Selectați ramura dorită, alegeți scan_depth=standard pentru utilizarea curentă sau extended pentru investigații. Profilul inițial păstrează aceeași acoperire de bază; inputul este rezervat extinderii controlate fără schimbarea interfeței operatorului.

Clasificarea constatărilor

Stare	Acțiune
Confirmată	issue cu severitate, proprietar, termen și dovadă de retest

Stare	Acțiune
Fals pozitiv	regulă/excludere minimă, comentată și revizuită
Risc acceptat	justificare, compensating control, aprobator și expirare
Remediată	link commit/PR și rezultat curat la reexecutare
Necunoscută	escaladare la dezvoltator/owner; nu se suprimă

Incident de secret

Dacă Gitleaks găsește un secret valid: revocați sau rotiți imediat secretul, investigați utilizarea, eliminați-l din sursă și tratați separat rescrierea istoricului. Ștergerea simplă din ultimul commit nu îl invalidează.

6. SonarQube Community Build self-hosted

Stackul include SonarQube și PostgreSQL pe o rețea internă. PostgreSQL nu este publicat. Interfața SonarQube este legată pe loopback și trebuie expusă, când este necesar, prin reverse proxy HTTPS.

Secvență de instalare

```
cp .env.example .env
editați parola CHANGE_ME
docker compose config
docker compose up -d
docker compose ps
```

Integrarea cu GitHub

Creați proiectul și un token dedicat fără drepturi administrative. Salvați SONAR_HOST_URL și SONAR_TOKEN în GitHub Actions Secrets. Dacă SonarQube este intern, folosiți un runner self-hosted aprobat; un runner GitHub-hosted nu poate ajunge la rețeaua privată.

Backup și restaurare

Realizați un pg_dump consistent și salvați volumele data/extensions/logs conform politicii. Testați restaurarea trimestrial într-un mediu izolat. Nu ștergeți volumele cu docker compose down -v în operațiunile normale.

7. Trecerea la profil blocant

După 2-4 săptămâni de execuții manuale și clasificarea fals-pozitivelor, controalele pot fi activate incremental. Ordinea recomandată este: secret valid, build/teste, vulnerabilități CRITICAL, SAST ERROR confirmat, lint pe cod nou, coverage pe cod nou, licențe deny și abia apoi controale extinse.

Fază	Trigger	Blocare
Pilot v1	workflow_dispatch	nu
Observare	pull_request opțional	nu; rapoarte
Gate critic	pull_request	secrete + build + CRITICAL
Gate matur	pull_request + release	politici calibrate și aprobate

8. Checklist de acceptare

- ☐ Există numai workflow_dispatch în workflow-ul auditului.
- ☐ permissions este contents: read.
- ☐ CI-ul și deploymentul preexistente nu au fost modificate.
- ☐ Semgrep, Trivy, Gitleaks, MegaLinter și SBOM produc artefacte.
- ☐ Retenția artefactelor este definită.
- ☐ Nu există tokenuri, IP-uri interne sau parole reale.
- ☐ YAML, shell și Docker Compose trec validarea sintactică.
- ☐ Rollback-ul este documentat.
- ☐ Branch Protection nu cere încă acest workflow.

Anexe - cod complet

Anexele următoare reproduc fișierele versionate. Liniile sunt numerotate, comentariile sunt verzi, iar cheile YAML și secțiunile sunt evidențiate cu albastru. Valorile CHANGE_ME sunt intenționat fictive.

Standard recomandat enterprise pentru replicare

Fișier: scripts-and-utlites-for-admins/devsecops-ci/docs/RECOMMENDED-ENTERPRISE-STACK.md | Limbaj: Markdown

#	COD SURSĂ
1	# Versiunea recomandată pentru replicare: `recommended-enterprise-v1`
2	
3	## Decizie
4	
5	> **SonarQube Community + Semgrep CE + MegaLinter + Trivy + Gitleaks +
6	> Reviewdog** reprezintă baza standard. Se adaugă **OSS Review Toolkit (ORT)**
7	> pentru conformitate, licențe și SBOM și **PR-Agent + Ollama** pentru review AI
8	> complet self-hosted. Stackul se distribuie printr-un workflow reutilizabil și
9	> devine obligatoriu pentru toate repository-urile după etapa de calibrare.
10	
11	## Componente
12	
13	Componentă Statut Responsabilitate
14	--- --- ---
15	SonarQube Community obligatoriu quality dashboard, maintainability, bugs, coverage
16	Semgrep CE obligatoriu SAST și reguli organizaționale proprii
17	MegaLinter obligatoriu code style și compliance multi-limbaj
18	Trivy obligatoriu CVE, imagini, repository, secrete și IaC
19	Gitleaks obligatoriu secrete în fișiere și istoricul Git
20	Reviewdog obligatoriu în faza PR comentarii inline din rezultatele deterministe
21	ORT obligatoriu pentru release/compliance licențe, copyright, policy-as-code, SBOM
22	PR-Agent + Ollama recomandat review AI self-hosted, exclusiv consultativ
23	
24	## Model de replicare
25	
26	1. Repository-ul central menține workflow-ul reutilizabil, versiunile și politicile.
27	2. Fiecare repository consumator fixează workflow-ul la un tag aprobat, nu la `main`.
28	3. Profilul pornește ca `manual-nonblocking-v1` pentru 2-4 săptămâni.
29	4. După triere și baseline, se activează `pull_request` în mod informativ.
30	5. Controalele deterministe critice devin Required Status Checks.
31	6. ORT rulează la release și periodic; politica de licențe este aprobată juridic.
32	7. PR-Agent rulează self-hosted cu Ollama și nu decide merge-ul.
33	
34	## Ordinea de activare a blocării
35	
36	1. secrete confirmate;
37	2. build și teste;
38	3. vulnerabilități CRITICAL remediabile;
39	4. reguli Semgrep `ERROR` confirmate;
40	5. Quality Gate SonarQube pe cod nou, unde capabilitatea este disponibilă;
41	6. linters pe cod nou/modificat;
42	7. policy ORT pentru licențe `deny` și `unknown`;
43	8. praguri HIGH și coverage după aprobarea proiectului.
44	

#	COD SURSĂ
45	Reviewdog doar publică rezultatele. PR-Agent + Ollama rămâne consultativ și nu este
46	niciodată singurul control care blochează integrarea.
47	
48	## Regula pentru repository-uri noi
49	
50	Orice repository nou trebuie onboardat cel puțin cu `manual-nonblocking-v1`.
51	Ținta organizațională este `recommended-enterprise-v1`. Excluderea unui control
52	obligatoriu necesită justificare, proprietar, control compensator, aprobator și
53	dată de expirare.
54	

Workflow GitHub generic - audit manual

Fișier: scripts-and-utiles-for-admins/devsecops-ci/templates/github/manual-devsecops-generic.yml | Limbaj: YAML

#	COD SURSĂ
1	name: Manual DevSecOps Audit
2	
3	on:
4	workflow_dispatch:
5	inputs:
6	scan_depth:
7	description: Audit depth
8	required: true
9	default: standard
10	type: choice
11	options: [standard, extended]
12	
13	permissions:
14	contents: read
15	
16	jobs:
17	semgrep:
18	runs-on: ubuntu-latest
19	container: semgrep/semgrep:latest
20	steps:
21	- uses: actions/checkout@v4
22	- run: semgrep scan --config auto --sarif --output semgrep.sarif .
23	continue-on-error: true
24	- uses: actions/upload-artifact@v4
25	if: always()
26	with: {name: semgrep-report, path: semgrep.sarif}
27	
28	trivy:
29	runs-on: ubuntu-latest
30	steps:
31	- uses: actions/checkout@v4
32	- uses: aquasecurity/trivy-action@master
33	with:
34	scan-type: fs
35	scanners: vuln,secret,misconfig
36	severity: HIGH,CRITICAL
37	format: sarif
38	output: trivy.sarif
39	exit-code: '0'
40	- uses: actions/upload-artifact@v4
41	if: always()
42	with: {name: trivy-report, path: trivy.sarif}
43	
44	gitleaks:
45	runs-on: ubuntu-latest
46	steps:
47	- uses: actions/checkout@v4
48	with: {fetch-depth: 0}
49	- name: Gitleaks history audit

#	COD SURSĂ
50	run: docker run --rm -v "\$PWD:/repo" zricethezav/gitleaks:latest detect --source=/repo --report-format=sarif --report-path=/repo/gitleaks.sarif --exit-code=0
51	- uses: actions/upload-artifact@v4
52	if: always()
53	with: {name: gitleaks-report, path: gitleaks.sarif, retention-days: 14}

Politică Semgrep custom

Fișier: scripts-and-utlities-for-admins/devsecops-ci/config/.semgrep.yml | Limbaj: YAML

#	COD SURSĂ
1	<code>rules:</code>
2	<code>- id: ernu-no-shell-true-suppression</code>
3	<code>message: Nu ascunde eșecul unui control de securitate cu `   true`.</code>
4	<code>severity: WARNING</code>
5	<code>languages: [bash]</code>
6	<code>patterns:</code>
7	<code>- pattern-regex: '(semgrep trivy gitleaks).*\ \\ s*true'</code>
8	<code>- id: ernu-no-hardcoded-private-key</code>
9	<code>message: Material de cheie privată detectat în cod.</code>
10	<code>severity: ERROR</code>
11	<code>languages: [generic]</code>
12	<code>pattern-regex: '-----BEGIN (RSA EC OPENSSH)?PRIVATE KEY-----'</code>
13	

Configurație MegaLinter

Fișier: scripts-and-utilities-for-admins/devsecops-ci/config/.mega-linter.yml | Limbaj: YAML

#	COD SURSĂ
1	APPLY_FIXES: none
2	VALIDATE_ALL_CODEBASE: true
3	SHOW_ELAPSED_TIME: true
4	PRINT_ALPACA: false
5	REPORT_OUTPUT_FOLDER: megalinter-reports
6	DISABLE_LINTERS:
7	- REPOSITORY_TRIVY
8	- REPOSITORY_GITLEAKS
9	MARKDOWN_MARKDOWNLINT_FILTER_REGEX_EXCLUDE: "(^ /)vendor/ ^ /)node_modules/ ^ /)artifacts/"
10	

Politică Gitleaks

Fișier: scripts-and-utlities-for-admins/devsecops-ci/config/gitleaks.toml | Limbaj: TOML

#	COD SURSĂ
1	title = "ERNU DevSecOps Gitleaks policy"
2	
3	[extend]
4	useDefault = true
5	
6	[[allowlists]]
7	description = "Documented fake values and fixtures"
8	paths = [
9	'^ /\\.env\\.example\$',
10	'^ /fixtures?/',
11	'^ /test-data/'
12	]
13	regexes = [
14	'CHANGE_ME',
15	'example\\.invalid',
16	'192\\.0\\.2\\.([0-9]{1,3})'
17	]
18	

Docker Compose SonarQube + PostgreSQL

Fișier: scripts-and-utlities-for-admins/devsecops-ci/platform/sonarqube/docker-compose.yml | Limbaj: YAML

#	COD SURSĂ
1	<code>services:</code>
2	<code>sonarqube:</code>
3	<code>image: sonarqube:community</code>
4	<code>restart: unless-stopped</code>
5	<code>depends_on:</code>
6	<code>db:</code>
7	<code>condition: service_healthy</code>
8	<code>environment:</code>
9	<code>SONAR_JDBC_URL: jdbc:postgresql://db:5432/\${SONAR_DB_NAME}</code>
10	<code>SONAR_JDBC_USERNAME: \${SONAR_DB_USER}</code>
11	<code>SONAR_JDBC_PASSWORD: \${SONAR_DB_PASSWORD}</code>
12	<code>ports:</code>
13	<code>- "127.0.0.1:\${SONAR_HTTP_PORT:-9000}:9000"</code>
14	<code>volumes:</code>
15	<code>- sonarqube_data:/opt/sonarqube/data</code>
16	<code>- sonarqube_extensions:/opt/sonarqube/extensions</code>
17	<code>- sonarqube_logs:/opt/sonarqube/logs</code>
18	<code>networks: [sonar]</code>
19	
20	<code>db:</code>
21	<code>image: postgres:17-alpine</code>
22	<code>restart: unless-stopped</code>
23	<code>environment:</code>
24	<code>POSTGRES_DB: \${SONAR_DB_NAME}</code>
25	<code>POSTGRES_USER: \${SONAR_DB_USER}</code>
26	<code>POSTGRES_PASSWORD: \${SONAR_DB_PASSWORD}</code>
27	<code>healthcheck:</code>
28	<code>test: ["CMD-SHELL", "pg_isready -U \${SONAR_DB_USER} -d \${SONAR_DB_NAME}"]</code>
29	<code>interval: 10s</code>
30	<code>timeout: 5s</code>
31	<code>retries: 10</code>
32	<code>volumes:</code>
33	<code>- sonarqube_db:/var/lib/postgresql/data</code>
34	<code>networks: [sonar]</code>
35	
36	<code>networks:</code>
37	<code>sonar:</code>
38	<code>internal: true</code>
39	
40	<code>volumes:</code>
41	<code>sonarqube_data:</code>
42	<code>sonarqube_extensions:</code>
43	<code>sonarqube_logs:</code>
44	<code>sonarqube_db:</code>
45	

Variable exemplu SonarQube

Fișier: scripts-and-utilites-for-admins/devsecops-ci/platform/sonarqube/.env.example | Limbaj: dotenv

#	COD SURSĂ
1	SONAR_HTTP_PORT=9000
2	SONAR_DB_NAME=sonarqube
3	SONAR_DB_USER=sonarqube
4	SONAR_DB_PASSWORD=CHANGE_ME_USE_A_LONG_RANDOM_PASSWORD
5	

Script validare configurații

Fișier: scripts-and-utillites-for-admins/devsecops-ci/scripts/validate-configs.sh | Limbaj: Bash

#	COD SURSĂ
1	<code>#!/usr/bin/env bash</code>
2	<code>set -Eeuo pipefail</code>
3	
4	<code>root_dir="\$(cd "\$(dirname "\${BASH_SOURCE[0]}")/.." && pwd)"</code>
5	
6	<code>python3 - <<'PY' "\$root_dir"</code>
7	<code>import pathlib, sys</code>
8	<code>import yaml</code>
9	<code>root = pathlib.Path(sys.argv[1])</code>
10	<code>for path in root.rglob("*.yaml"):</code>
11	<code>with path.open(encoding="utf-8") as handle:</code>
12	<code>yaml.safe_load(handle)</code>
13	<code>print(f"YAML OK: {path.relative_to(root)}")</code>
14	<code>PY</code>
15	
16	<code>docker compose --env-file "\$root_dir/platform/sonarqube/.env.example" \</code>
17	<code>-f "\$root_dir/platform/sonarqube/docker-compose.yaml" config --quiet</code>
18	<code>echo "Compose OK"</code>
19	

Referință operațională finală

Sursa autoritativă pentru reutilizare este directorul devsecops-ci din repository-ul scripts-and-utillies-for-admins. Cererea viitoare „aplică aceeași verificare” înseamnă profilul manual-nonblocking-v1, până când proprietarul aprobă explicit un profil automat sau blocant.

ERNU.EU

IT & Security Solutions - document tehnic pregătit pentru utilizare operațională și versionare în GitHub.